Date: 19-03-2026

Font

Font stacks:

font stack so that the browser has multiple fonts it can choose from. This involves a font-family value consisting of multiple font names separated by commas,

e.g.,

```
p {
  font-family: "Trebuchet MS", "Verdana", sans-serif;
}
p {
  color: red;
  font-family: "Helvetica", "Arial", sans-serif;
}
```

Font size:

Font size is set with font-size property.

- px (pixels): The number of pixels high you want the text to be. This is an absolute unit — it results in the same final computed value for the font on the page in pretty much any situation.
- ems: 1 em is equal to the font size set on the parent element of the current element we are styling (more specifically, the width of a capital letter M contained inside the parent element). This can become tricky to work out if you have a lot of nested elements with different font sizes set, but it is doable, as you'll see below. Why bother? It is quite natural once you get used to it, and you can use em to size everything, not just text. You can have an entire website sized using em, which makes maintenance easy.
- rems: These work just like em, except that 1 rem is equal to the font size set on the root element of the document (i.e., <html>), not the parent element. This makes doing the maths to work out your font sizes much easier.

The font-size of an element is inherited from that element's parent element.

the standard font-size of which is set to 16px across browsers.

Font style, font weight, text transform, and text decoration:

font-style:

Used to turn italic text on or off. Possible values are as follows

- normal: Sets the text to the normal font (turns existing italics off).
- italic: Sets the text to use the italic version of the font, if available; if not, it will simulate italics with oblique instead.
- oblique: Sets the text to use a simulated version of an italic font, created by slanting the normal version.

font-weight:

Sets how bold the text is. This has many values available in case you have many font variants available (such as *-light*, *-normal*, *-bold*, *-extrabold*, *-black*, etc.), but realistically you'll rarely use any of them except for normal and bold:

- normal, bold: Normal and bold font weight.
- lighter, bolder: Sets the current element's boldness to be one step lighter or heavier than its parent element's boldness.
- 100 – 900: Numeric boldness values that provide finer grained control than the above keywords, if needed.

text-transform:

- none: Prevents any transformation.
- uppercase: Transforms all text to capitals.
- lowercase: Transforms all text to lower case.
- capitalize: Transforms all words to have the first letter capitalized.
- full-width: Transforms all glyphs to be written inside a fixed-width square, similar to a monospace font, allowing aligning of, e.g., Latin characters along with Asian language glyphs (like Chinese, Japanese, Korean).

text-decoration:

Sets/unsets text decorations on fonts (you'll mainly use this to unset the default underline on links when styling them). Available values are:

- none: Unsets any text decorations already present.
- underline: Underlines the text.
- overline: Gives the text an overline.
- line-through: Puts a strikethrough over the text.

You should note that text-decoration can accept multiple values at once if you want to add multiple decorations simultaneously, for example, text-decoration: underline overline. Also

note that text-decoration is a shorthand property for text-decoration-line, text-decoration-style, and text-decoration-color. You can use combinations of these property values to create interesting effects, for example: text-decoration: line-through red wavy.

Text drop shadows:

You can apply drop shadows to your text using the text-shadow property. This takes up to four values, as shown in the example below:

text-shadow: 4px 4px 5px red;

The four properties are as follows:

1. The horizontal offset of the shadow from the original text — this can take most available CSS length and size units, but you'll most commonly use px; positive values move the shadow right, and negative values left. This value has to be included.
2. The vertical offset of the shadow from the original text. This behaves similarly to the horizontal offset, except that it moves the shadow up/down, not left/right. This value has to be included.
3. The blur radius: a higher value means the shadow is dispersed more widely. If this value is not included, it defaults to 0, which means no blur. This can take most available CSS length and size units.
4. The base color of the shadow, which can take any CSS color unit. If not included, it defaults to currentColor, i.e., the shadow's color is taken from the element's color property.

Multiple shadows:

We can apply multiple shadows to the same text by including multiple shadow values separated by commas, for example:

```
h1 {
  text-shadow:
    1px 1px 1px red,
    2px 2px 1px red;
}
```

Text layout:

With basic font properties out of the way, let's have a look at properties we can use to affect text layout.

Text alignment:

The text-align property is used to control how text is aligned within its containing content box. The available values are listed below. They work in pretty much the same way as they do in a regular word processor application:

- left: Left-justifies the text.
- right: Right-justifies the text.
- center: Centers the text.
- justify: Makes the text spread out, varying the gaps in between the words so that all lines of text are the same width. You need to use this carefully — it can look terrible, especially when applied to a paragraph with lots of long words in it. If you are going to use this, you should also think about using something else along with it, such as hyphens, to break some of the longer words across lines.

Line height:

The line-height property sets the height of each line of text

This property can not only take most length and size units, but can also take a unitless value, which acts as a multiplier and is generally considered the best option. With a unitless value, the font-size gets multiplied and results in the line-height.

The recommended line height is around 1.5 – 2 (double spaced).

```
p {
  line-height: 1.6;
}
```

Letter and word spacing:

The letter-spacing and word-spacing properties allow you to set the spacing between letters and words in your text. You won't use these very often, but might find a use for them to obtain a specific look, or to improve the legibility of a particularly dense font. They can take most length units.

```
p::first-line {
  letter-spacing: 4px;
  word-spacing: 4px;
}
```

Other properties:

Font styles:

- font-variant: Switch between small caps and normal font alternatives.

- font-kerning: Switch font kerning options on and off.
- font-feature-settings: Switch various OpenType font features on and off.
- font-variant-alternates: Control the use of alternate glyphs for a given font-face.
- font-variant-caps: Control the use of alternate capital glyphs.
- font-variant-east-asian: Control the usage of alternate glyphs for East Asian scripts, like Japanese and Chinese.
- font-variant-ligatures: Control which ligatures and contextual forms are used in text.
- font-variant-numeric: Control the usage of alternate glyphs for numbers, fractions, and ordinal markers.
- font-variant-position: Control the usage of alternate glyphs of smaller sizes positioned as superscript or subscript.
- font-size-adjust: Adjust the visual size of the font independently of its actual font size.
- font-stretch: Switch between possible alternative stretched versions of a given font.
- text-underline-position: Specify the position of underlines set using the text-decoration-line property underline value.
- text-rendering: Try to perform some text rendering optimization.

Text layout styles:

- text-indent: Specify how much horizontal space should be left before the beginning of the first line of the text content.
- text-overflow: Define how overflowed content that is not displayed is signaled to users.
- white-space: Define how whitespace and associated line breaks inside the element are handled.
- word-break: Specify whether to break lines within words.
- direction: Define the text direction. (This depends on the language and usually it's better to let HTML handle that part as it is tied to the text content.)
- hyphens: Switch on and off hyphenation for supported languages.
- line-break: Relax or strengthen line breaking for Asian languages.
- text-align-last: Define how the last line of a block or a line, right before a forced line break, is aligned.
- text-orientation: Define the orientation of the text in a line.
- overflow-wrap: Specify whether or not the browser may break lines within words in order to prevent overflow.

- writing-mode: Define whether lines of text are laid out horizontally or vertically and the direction in which subsequent lines flow.

Font shorthand:

Many font properties can also be set through the shorthand property font. These are written in the following order: font-style, font-variant, font-weight, font-stretch, font-size, line-height, and font-family.

Among all those properties, only font-size and font-family are required when using the font shorthand property.

Example:

font:

italic normal bold normal 3em/1.5 "Helvetica",

"Arial",

sans-serif;

Styling Lists and Links:

- The <ul> and <ol> elements have a top and bottom margin of 16px (1em) and a padding-left of 40px (2.5em). If the directionality attribute dir is set to right-to-left (rtl) for ul and ol elements, in that case padding-right comes into effect and its default value is 40px (2.5em).
- The list items (<li> elements) have no set defaults for spacing.
- The <dl> element has a top and bottom margin of 16px (1em), but no padding set.
- The <dd> elements have margin-left of 40px (2.5em).
- The <p> elements we've included for reference have a top and bottom margin of 16px (1em) — the same as the different list types.

Handling list spacing:

When styling lists (ul, ol, dl), we need to:

- Keep **same vertical spacing** (called *vertical rhythm*)
- Keep **consistent font sizes and spacing** with paragraphs and headings

This makes a page look clean, balanced and readable.

Rule 1: Base font setup:

html {

font-family: "Helvetica", "Arial", sans-serif;

font-size: 10px;

```
}
```

All elements inherits this font.

Rule 2: Heading size:

```
h2 {
  font-size: 2rem;
}
```

- 2rem = 2 × 10px = 20px
- Makes headings **larger than normal text**

Rule 3: Paragraphs and Lists same size:

```
ul,
ol,
dl,
p {
  font-size: 1.5rem;
}
```

Sets same font for paragraphs and list to make them looks uniform and it helps to maintain vertical rhythm.

Rule 4: Line Spacing:

```
li,
p {
  line-height: 1.5;
}
```

Setting line spacing helps to improve readability and same spacing everywhere.

Rule 5: Description List spacing:

```
dd,
dt {
  line-height: 1.5;
}
```

It keeps consistent spacing like paragraphs.

Rule 6: Bold Terms:

```
dt {
  font-weight: bold;
}
```

It helps them stand out from descriptions.

List Specific styles:

List specific styles:

There are three properties that sets on the <ul> or <ol> elements.

- list-style-type: Sets the type of bullets to use for the list, for example, square or circle bullets for an unordered list, or numbers, letters, or roman numerals for an ordered list.
- list-style-position: Sets whether the bullets, at the start of each item, appear inside or outside the lists.
- list-style-image: Allows you to use a custom image for the bullet, rather than a simple square or circle.

Bullet styles:

list-style-type property allows you to set what type of bullet to use for the bullet points.

```
ol {
  list-style-type: upper-roman;
}
```

With this we can specify the style type for the list like roman, numbers, etc.

Bullet Position:

The list-style-position property sets whether the bullets appear inside the list items, or outside them before the start of each item. The default value is outside, which causes the bullets to sit outside the list items, as seen above.

```
ol {
  list-style-type: upper-roman;
  list-style-position: inside;
}
```

Using custom bullet image:

The list-style-image property allows you to use a custom image for your bullet.

```
ul {
  list-style-image: url("star.svg");
```

```
}
```

list-style shorthand:

Three properties mentioned above can be set using a single property

```
ul {
  list-style-type: square;
  list-style-image: url("example.png");
  list-style-position: inside;
}
```

This can be replaced with:

```
ul {
  list-style: square url("example.png") inside;
}
```

Controlling list counting:

Start:

The start attribute allows you to start the list counting from a number other than 1.

```
<ol start="4">
  <li>Toast pita, leave to cool, then slice down the edge.</li>
  <li>
    Fry the halloumi in a shallow, non-stick pan, until browned on both sides.
  </li>
  <li>Wash and chop the salad.</li>
  <li>Fill pita with salad, hummus, and fried halloumi.</li>
</ol>
```

Reversed:

The reversed attribute will start the list counting down instead of up.

```
<ol start="4" reversed>
  <li>Toast pita, leave to cool, then slice down the edge.</li>
  <li>
    Fry the halloumi in a shallow, non-stick pan, until browned on both sides.
```

```
  </li>
  <li>Wash and chop the salad.</li>
  <li>Fill pita with salad, hummus, and fried halloumi.</li>
</ol>
```

Value:

The value attribute allows you to set your list items to specific numerical values.

```
<ol>
  <li value="2">Toast pita, leave to cool, then slice down the edge.</li>
  <li value="4">
    Fry the halloumi in a shallow, non-stick pan, until browned on both sides.
  </li>
  <li value="6">Wash and chop the salad.</li>
  <li value="8">Fill pita with salad, hummus, and fried halloumi.</li>
</ol>
```

Styling links:

Link states:

Different states that the links can exist in and that are styling using different pseudo-classes

- **Link**: A link that has a destination (i.e., not just a named anchor), styled using the :link pseudo class.
- **Visited**: A link that has already been visited (exists in the browser's history), styled using the :visited pseudo class.
- **Hover**: A link that is hovered over by a user's mouse pointer, styled using the :hover pseudo class.
- **Focus**: A link that is focused (e.g., moved to by a keyboard user using the Tab key or something similar, or programmatically focused using HTMLElement.focus()) — this is styled using the :focus pseudo class.
- **Active**: A link that is activated (for example, clicked on), styled using the :active pseudo class.

Default styles:

- Links are underlined.
- Unvisited links are blue.

- Visited links are purple.
- Hovering a link makes the mouse pointer change to a little hand icon.
- Focused links have an outline around them — you should be able to focus on the links on this page with the keyboard by pressing the tab key.
- Active links are red.

The default styles can be turned off/changed using the following CSS properties:

- color for the text color.
- cursor for the mouse pointer style — you shouldn't turn this off unless you've got a very good reason.
- outline for the text outline. An outline is similar to a border. The only difference is that a border takes up space in the box, and an outline doesn't: outlines sit on top of the background. The outline is a useful accessibility aid, so it should not be removed without adding another method of indicating the focused link.

Styling links:

```
a {

}


a:link {

}


a:visited {

}


a:focus {

}


a:hover {

}


a:active {

}
```

This order is important because link styles build on one another. For example, the styles in the first rule will apply to all the subsequent ones. When a link is activated, it's usually also hovered over. If you put these in the wrong order, and you're changing the same properties in each ruleset, things won't work as you expect. To remember the order, you could try using a mnemonic like **L**o**V**e **F**ears **HA**te.

Including Icons on Links:

A common practice is to include icons on links to indicate what kind of content the link points to.

```
body {
  width: 300px;
  margin: 0 auto;
  font-family: sans-serif;
}

a[href^="http"]::after {
  content: "";
  display: inline-block;
  width: 0.8em;
  height: 0.8em;
  margin-left: 0.25em;

  background-size: 100%;
  background-image: url("external-link-52.png");
}
```

a[href^="http"] selects <a> elements, but only if they have an href attribute with a value that begins with "http".

Here in this example we used pseudo elements that the links starting with http after that link we added a icon.

Styling links as buttons:

states like hover can be used to style many different elements, not just links — you might want to style the hover state of paragraphs, list items, or other things.

In addition, links are commonly styled to behave like buttons. A website navigation menu can be marked up as a set of links, and this can be styled to look like a set of control buttons or tabs that provide the user with access to other parts of the site.

```
<nav class="container">
  <a href="#">Home</a>
  <a href="#">Pizza</a>
  <a href="#">Music</a>
  <a href="#">Wombats</a>
  <a href="#">Finland</a>
</nav>
body,
html {
  margin: 0;
  font-family: sans-serif;
}

.container {
  display: flex;
  gap: 0.625%;
}

a {
  flex: 1;
  text-decoration: none;
  outline-color: transparent;
  text-align: center;
  line-height: 3;
  color: black;
}
```

```
a:link,
a:visited,
a:focus {
  background: palegoldenrod;
  color: black;
}

a:hover {
  background: orange;
}

a:active {
  background: darkred;
  color: white;
}
```

Web fonts:

CSS allows you to specify font files, available on the web, to be downloaded along with your website as it's accessed.

This means that any browser supporting this CSS feature can display the fonts you've specifically chosen.

@font-face ruleset at the start of the CSS, which specifies the font file(s) to download:

```
@font-face {
  font-family: "myFont";
  src: url("myFont.woff2");
}
```

Below this you use the font family name specified inside @font-face to apply your custom font to anything you like, as normal:

```
html {
  font-family: "myFont", "Bitstream Vera Serif", serif;
}
```

1. Fonts generally aren't free to use without restrictions. You have to pay for them and/or follow other license conditions, such as crediting the font creator in your code (or on your site). You shouldn't steal fonts and use them without giving proper credit.
2. All major browsers support WOFF/WOFF2 (Web Open Font Format versions 1 and 2). Even older browsers such as IE9 (released in 2011) support the WOFF format.

Adding our own web fonts:

By default, websites use **system fonts**, but you can add **custom fonts (web fonts)** to make your design look better.

Finding fonts:

Types of font sources:

1. Free font websites
2. Paid font websites
3. Online font services

Downloading Fonts

- Download fonts (usually ZIP files)
- Extract (unzip) them
- Inside you'll find:
    - .ttf or .otf files
    - Different styles (bold, italic, etc.)

Generating web font code:

Browsers need special formats like:

- .woff
- .woff2

To create this we can use transfonter webfont Generator

Steps:

1. Upload font
2. Convert
3. Download ZIP
4. Extract files

4. Project Setup

- Put fonts in a folder (e.g., fonts/)

The project should be like this:

```
project/
  ├── index.html
  ├── style.css
 └── fonts/
```

Using @font-face

This should be added at top of our css:

```
@font-face {
  font-family: "myFont";
  src: url("fonts/font.woff2") format("woff2"),
       url("fonts/font.woff") format("woff");
}
```

Using Online Font Services (Easier Method)

Example: **Google Fonts**

Steps:

1. Go to Google Fonts
2. Select font
3. Copy <link>
4. Paste in HTML <head>
5. Use font in CSS

CSS Layout basics:

CSS page layout techniques allow us to take elements contained in a web page and control where they're positioned relative to the following factors: their default position in normal layout flow, the other elements around them, their parent container, and the main viewport/window.

Normal layout flow:

Elements on a webpage lay out in **normal flow** if you haven't applied any CSS to change the way they behave.

How are elements laid out by default:

The process begins as the boxes of individual elements are laid out in such a way that any padding, border, or margin they happen to have is added to their content. This is what we call the box model.

In html every element is treated like a box.

The box has:

Content → Padding → Border → Margin

Block level elements behaviour -> By default, block elements (like , <p>) take full width of parent

They grow vertically to fit content.

Inline elements behaviour -> Inline elements (like , <a>) only take space needed for content

They do not start on a new line.

Special case (some inline elemenst with size) -> Some inline elements (like <img>) can have:

- width
- height

But they are still inline elements.

Normal Layout Flow

This is how elements are arranged by default.

Block elements:

- Appear one below another
- Follow vertical flow

Top → Bottom

Inline elements:

- Appear side by side
- Wrap to next line if needed

Left → Right → Next line

Margin collapsing:

When two vertical elements have margins:

div1 { margin-bottom: 20px; }

div2 { margin-top: 30px; }

we might expect 20px + 30px = 50px

but actual result will be only 30px the 20px will disappear this is called margin collapsing.

It only happens in vertical direction not in horizontal direction.

Overriding normal flow:

The display property

Standard values such as block, inline or inline-block can change how elements behave in normal flow, for example, by making a block-level element behave like an inline-level element

Floats

Applying a float value such as left can cause block-level elements to wrap along one side of an element, like the way images sometimes have text floating around them in magazine layouts.

Positioning

The position property allows you to precisely control the placement of boxes inside other boxes. static positioning is the default in normal flow, but you can cause elements to be laid out differently using other values, for example, fixing them to the top of the browser viewport using position: fixed.

Specific layout systems accessed through display

We also have entire layout methods that are enabled via specific display values. The most important ones for you to know about are CSS grid and Flexbox, which both alter how child elements are laid out inside their parents.

Responsive design

Responsive design refers to creating layouts that adapt to different devices the web page is rendered on (for example, desktops and mobile phones). Responsive design doesn't provide any specific layout tools of its own; its most significant component is the @media at-rule, which allows you to apply different layouts depending on device attributes such as screen width or resolution.

Other layout techniques:

Table layout

Features designed for styling parts of an HTML table can be used on non-table elements using display: table and associated properties.

Multi-column layout

The multi-column layout properties can cause the content of a block to lay out in columns, as you might see in a newspaper.

Floats:

The background of floats:

The float property was introduced to allow web developers to implement layouts involving an image floating inside a column of text, with the text wrapping around the left or right of it. The kind of thing you might get in a newspaper layout.

But web developers quickly realized that you can float anything, not just images, so the use of float broadened.